

```

#include <iostream>
# include <cstdlib>
#include <ctime>

# define SIZE 100

using namespace std;

class Node
{
public:
Node()
{
    next = NULL;
    pre = NULL;
}

Node(int n)
{
    data = n;
    next = NULL;
    pre = NULL;
}

int getData()
{
    return data;
}
Node *getNext()
{
    return next;
}
Node *getpre()
{
    return pre;
}
void setData(int d)
{
    data = d;
}
void setNext(Node *n)
{
    next = n;
}

void setPre(Node* p)
{
    pre = p;
}

private:

int data;
Node *next, *pre;

};

class List

```

```

{
public:

List()
{
    list = NULL;
}

List(int n)
{
    generate(n);
}

void generate(int n)
{

    int j;
    list = NULL;
    for (j = 0; j < n; j++)
        generate();
}

void generate()
{
    Node *buf = new Node(rand());
    buf->setNext(list);
    if (list != NULL)
        list->setPre(buf);
    list = buf ;
}

void bubbleSort()
{
    if (list == NULL)
        return;

    bool swapped ;

    do
    {
        swapped = false;
        Node *cur = list;

        while (cur->getNext() != NULL)
        {

            if (cur->getData() > cur->getNext()->getData())
            {
                //swap data
                int temp = cur->getData();
                cur->setData(cur->getNext()->getData());
                cur->getNext()->setData(temp);
                swapped = true;
            }

        }
    }
}

```

```

        cur = cur->getNext();
    }
} while (swapped);
}

void selectionSort()
{
    Node *cur = list;

    while(cur != NULL)
    {
        Node *min = cur;
        Node *nextNode = cur ->getNext();

        while (nextNode != NULL)
        {
            if (nextNode->getData() < min->getData())
            {
                min = nextNode;
            }
            nextNode = nextNode->getNext();
        }

        //swap cur and min
        if (cur != min)
        {
            int temp = cur->getData();
            cur->setData(min->getData());
            min->setData(temp);
        }
        cur = cur->getNext();
    }
}

```

```

void insertionSort()
{
    if (list == NULL)
        return;

    Node *sorted = NULL;
    Node *cur = list;

    while (cur != NULL)
    {
        Node *nextNode = cur->getNext();

        if (sorted == NULL || sorted->getData() >= cur ->getData())
        {
            cur->setNext(sorted);
            if(sorted != NULL)
                sorted->setPre(cur);
        }
    }
}

```

```

        sorted = cur;
        sorted->setPre(NULL);
    }

    else
    {
        Node *temp = sorted;
        while (temp->getNext() != NULL && temp->getNext()->getData() < cur->getData())
        {
            temp = temp->getNext();
        }

        cur->setNext(temp->getNext());

        if (temp->getNext() != NULL)
            temp->getNext()->setPre(cur);

        temp->setNext(cur);
        cur->setPre(temp);
    }

    cur = nextNode;
}
list = sorted;
}

void print()
{
    Node *cur = list;
    while (cur != NULL)
    {
        cout << cur->getData() << " ";
        cur = cur->getNext();
    }

    cout << endl;
}

private:
Node *list;

};

int main()
{
    srand(time(NULL));
    List *l = new List(10);
    l->print();
    l->bubbleSort();
    l->print();

    l = new List(10);
    l->print();
    l->insertionSort();
    l->print();
}

```

```
l = new List(10);  
l->print();  
l->selectionSort();  
l->print();
```

```
}
```